

PIDR 2022 - Projet n°42

Urbanloop, c'est du propre !

Tommy BARAFANI

Télécom Nancy

tommy.barafani@telecomnancy.eu

Théo CLEMENTZ

Télécom Nancy

theo.clementz@telecomnancy.eu

Thibault CHOLEZ

Télécom Nancy

thibault.cholez@loria.fr

Encadrant académique

Julien LANG

Télécom Nancy

julien.lang@univ-lorraine.fr

Encadrant industriel

Abstract—Notre projet a eu comme objectif l'ajout au logiciel de simulations du projet Urbanloop de la possibilité d'envoi en nettoyage et en révision des capsules d'un réseau. Nous avons utilisé Python pour apporter nos modifications de l'architecture du simulateur, ainsi que Javascript pour modifier la partie navigateur.

Nous avons d'abord établi les changements conceptuels du simulateur. Trois nouveaux types d'entrepôts ainsi que deux gestionnaires ont été implantés. Nous avons ensuite modifié l'interface utilisateur du logiciel, pour prendre en compte les changements apportés et les faire apparaître sur un réseau.

Nous avons finalement adapté plusieurs réseaux existants à la nouvelle architecture pour effectuer nos tests, et adapté l'application mobile pour qu'un utilisateur puisse signaler un pod comme à réviser ou à laver.

MOTS-CLÉS

- Python
- révision
- lavage
- Javascript
- Urbanloop
- réseau
- simulation
- transport

I. INTRODUCTION

A. Projet Urbanloop

Notre PIDR s'inscrit dans le cadre du projet Urbanloop. Ce projet a commencé en tant que projet pédagogique interne à l'ENSEM, une école ingénieur à Vandœuvre-lès-Nancy spécialisée dans l'électronique et la mécanique. Progressivement, le projet a pris de l'ampleur et s'est étendu à d'autres écoles d'ingénieur de Lorraine-INP (composante de l'Université de Lorraine). C'est à cet occasion que Telecom Nancy, notre école, s'est associée au projet, sur des problématiques dans son champ de compétence privilégié : informatique, algorithmique, électronique.

Urbanloop est un nouveau système de transport public urbain, destiné en particulier à des villes et des métropoles de la taille de la Métropole du Grand Nancy. Son principe est de promouvoir les transports publics en leur ajoutant la rapidité, la flexibilité et le confort d'un véhicule individuel. Un réseau Urbanloop consiste en de nombreux véhicules, appelés capsules, d'une capacité d'une à deux personnes, circulant

en conduite autonome sur un réseau dédié sans obstacles, formant des boucles à sens unique interconnectées, permettant d'atteindre n'importe quelle station de destination depuis n'importe quelle station de départ sans arrêt intermédiaire et sans correspondance. Le réseau lui-même est formé de rails apportant l'énergie (électrique) à la capsule, et peut-être en fonction des contraintes de l'environnement posé au sol ou enterré à faible profondeur, séparé de la surface par des plaques transparentes (sur lesquelles peuvent se trouver une piste cyclable, des traversées de route...).

L'utilisation du réseau par un voyageur est conçue pour être aussi fluide et personnalisée que le serait l'utilisation d'un véhicule personnel ou de location : le voyageur achète son ticket sur une application mobile ou à un automate en station, il le valide à une borne et une capsule se rend disponible et s'ouvre pour qu'il y prenne place. Les stations étant construites en dérivation par rapport au flux principal, l'objectif est d'avoir la plus grande partie du temps des capsules en station prêtes à accueillir immédiatement les voyageurs quand ils se présentent, à la manière de taxis en attente près d'un point d'intérêt. Quand le voyageur est à bord, la capsule se referme et entame son trajet, avec donc un seul passager. Ce trajet est direct, sans arrêt intermédiaires, correspondances ni feux rouges : toutes les boucles sont interconnectées, à sens unique, sans obstacles, et toutes les capsules sont pilotées au moyen d'algorithmes optimisant en permanence la fluidité de la circulation et les temps de trajet, par exemple en répartissant intelligemment la circulation entre les différentes boucles, en anticipant les croisements, en maintenant une vitesse identique entre des capsules qui se suivent, etc. La capsule finit par se détourner du flux principal pour arriver en station d'arrivée, où elle s'arrête et s'ouvre pour laisser sortir le voyageur, après un parcours qui aura idéalement duré l'équivalent du trajet en voiture correspondant, en heures creuses. La capsule est alors à nouveau disponible dans la station d'arrivée pour un autre trajet.

Le développement de ce système, dans tous ses aspects (mécanique, algorithmique, urbanistique, financier...) a commencé en 2017, et a notamment vu se succéder plusieurs prototypes de capsules, de rails et de systèmes de pilotage automatique. Pendant ce développement, un premier réseau de test, formé d'une boucle et d'une station a vu le jour à Brabois, près de l'ENSEM. Par la suite, un plus grand terrain

d'expérimentation a vu le jour à Tomblaine, également dans la banlieue de Nancy, circuit où a notamment été battu le record de la moindre consommation énergétique au kilomètre pour un véhicule autonome sur rails. Une entreprise a en parallèle été constituée (Urbanloop SA) dans le but de centraliser les développements et les investissements, et pour servir de support à un développement commercial. La démonstration à échelle 1 d'un réseau d'une boucle est prévu à l'occasion des Jeux Olympiques de Paris en 2024.

A Telecom Nancy, le travail sur le projet Urbanloop a commencé en 2018. Les deux principaux domaines de recherche qui y sont traités sont la conception et le test des algorithmes de pilotage autonome des capsules (ou plutôt du réseau), et des problématiques liées à l'électronique et l'informatique embarquée. C'est dans ce cadre qu'a été développé à l'école, par des projets pédagogiques réalisés par des étudiants, un simulateur de réseau Urbanloop, permettant de visualiser un réseau (virtuel) en fonctionnement, mesurer ses performances, ses coûts, et développer les algorithmes de pilotage des capsules. Ont également été ajoutés à ce réseau une application Android prototypant l'application destinée aux voyageurs, ainsi qu'une application Android prototypant le système intégré aux bornes en station, scannant les tickets des voyageurs pour leur ouvrir l'accès aux stations. Ces applications sont fonctionnelles, et permettent donc de simuler l'ensemble du parcours d'un voyageur, de l'achat du ticket à l'arrivée à destination.

Le simulateur a été développé dans le but de remplacer ou de compléter des circuits d'essais pour tout ce qui concerne l'aspect logiciel. Au stade où il était quand nous avons commencé à travailler sur le projet, le simulateur permettait la circulation d'un nombre arbitraire de capsules sur un réseau donné, avec des logiques de routage relativement simple (chaque capsule suit le plus court chemin vers sa destination, les boucles ont une vitesse de circulation constante), avec une connexion possible avec les applications Android, des simulations aléatoires de demande de trajet par des voyageurs fictifs, une visualisation en temps réel ou accéléré de l'état du réseau simulé, et des moyens d'obtenir des statistiques diverses pour mesurer les performances du réseau (temps de trajet moyen, vitesse moyenne, temps moyen d'attente en station, kilométrage parcouru, etc.).

Notre PIDR avait pour but d'y ajouter la gestion de l'entretien des capsules : entretien mécanique (révisions régulières) et lavage. Nous avons donc rendu possible l'ajout dans les réseaux simulés de points de lavage et de révision dans le réseau, et la gestion de l'envoi des capsules en révision ou au lavage par le système de pilotage, en fonction notamment de trois critères : le kilométrage parcouru par la capsule, le temps écoulé depuis le dernier entretien, ainsi que les signalements des usagers. Nous avons donc été amenés à intervenir tant sur le simulateur en lui-même que sur l'application Android destinée aux usagers, sur laquelle les signalements peuvent être faits.

A Telecom Nancy, un PIDR est un Projet interdisciplinaire, c'est-à-dire un projet de deuxième année à réaliser en groupe constitué pour une large part de programmation, visant à

répondre à un besoin d'une entreprise ou d'un laboratoire de recherche.

Notre PIDR a eu lieu après plusieurs autres projets.

- stage
Consolidation et extension du simulateur d'UrbanLoop
Tristan Le Godais
2018–2019
- stage
Interfaçage du simulateur global du réseau UrbanLoop avec celui des postes d'aiguillage
Malo Mongard
2018–2019
- Projet industriel
Simulation du réseau de transport urbain par capsules Urbanloop
Charlotte ASSELIN-BOULLÉ, Baptiste CHOCOT, Thibault HENRY
2018–2019
- stage
Extension de l'application mobile Urbanloop
Lilian Bidet
2019–2020
- stage
Fiabilisation et exploitation du simulateur d'UrbanLoop
Pierre-Antoine Grosselin
2019–2020
- Projet industriel
Extension et amélioration du simulateur de réseau Urbanloop
Valentin GAU, Antoine OLIVIER, Dylan ALIBAY
2019–2020
- Projet interdisciplinaire ou de recherche
Cahier des charges fonctionnel de l'application mobile d'Urbanloop
Liam Lyons, Timon Fugier
2019–2020
- Projet industriel
Réalisation d'une borne de station Urbanloop et exploitation du simulateur
Clément Crouzet, Tristan Deniau, Arthur Saout
2020–2021

II. ASPECTS TECHNIQUES DU SIMULATEUR

A. Fonctionnement du simulateur

Le simulateur vise à remplacer un circuit d'essai physique en simulant la circulation de capsules virtuelles sur des réseaux conçus à l'aide de d'éditeur de réseau (développé pendant un autre projet à l'école). L'évolution de l'état du réseau simulé peut être visualisé sous forme d'une page dans un navigateur. Il est possible d'interagir avec le simulateur via le navigateur, ou en ligne de commandes.

1) *Structure d'un réseau*: Un réseau est stocké sous format JSON, fichier qui sera la base de la construction de nombreux objets Python représentant les différents éléments du réseau. Il est également possible de sauvegarder l'état courant d'un

réseau sous format JSON ré-importable par la suite. (Note : dans la suite, les noms donnés aux différents éléments sont ceux des classes Python correspondantes, le lecteur pourra se référer au glossaire réalisé par MM. Le Godais et Mongard [4]). Un réseau est constitué d'une ou plusieurs boucles. Sur une boucle, les pods circulent dans un seul sens. Une boucle est principalement une succession linéaire d'objets Switch et d'objets Road. D'un Switch à un autre Switch, il peut y avoir à la fois un objet Road et un "pont", c'est-à-dire un objet Section suivi d'un objet Step suivi d'un objet Section. Cet aspect technique est détaillé en II-C. Cela permet d'introduire dans le réseau des objets Step, c'est-à-dire des objets Shed, Station et Sensor. Les objets Shed sont les entrepôts d'où partent les pods au début de la simulation. Les objets Station sont les stations où les voyageurs peuvent monter à bord des pods. Les objets Sensor permettent d'effectuer des tests.

Le simulateur fonctionne avec ou sans l'interface dans le navigateur. Nous avons utilisé cette dernière pendant tout le projet pour mieux visualiser le comportement de l'ensemble des composants.

Lors du lancement du simulateur avec interface Web :

- le serveur Flask est démarré (commande `python3 main -p`);
- pour chaque réseau chargé (un réseau peut être chargé ultérieurement), un objet Simulation ainsi que tous les objets dont il est composé sont créés (importation d'un fichier JSON via le bouton correspondant dans l'interface du navigateur) ;
- pour chaque objet Simulation créé, une boucle principale est lancée, elle correspond à l'attente de mise à jour de tous les composants de l'objet Simulation, et s'effectue tant que dure la simulation.

La liaison entre l'interface Web et la simulation est permise grâce à l'enregistrement des données de l'objet Simulation dans un dictionnaire au format JSON. En effet, lorsque des modifications ou des appels de méthodes modifient des données de l'objet Simulation dont dépend l'interface Web, une fonction utilisée par le serveur Flask (`_get_network()`) synchronisée avec les simulations est appelée, et appelle sur l'objet Simulation la méthode `serialize()`. Cette méthode renvoie un dictionnaire au format JSON qui est ensuite envoyé au serveur Flask et lu par les méthodes JavaScript chargées de mettre à jour les composants graphiques.

L'évolution du simulateur utilise un objet Environment du module Simpy. Elle est permise par la boucle principale, présente dans le fichier `app.py`. Dans cette boucle est appelée la méthode `update()` de l'objet Simulation correspondant à chaque simulation lancée. C'est cette méthode qui lance la mise à jour de tous les composants de la simulation.

Cette mise à jour utilise des messages. Ils sont envoyés d'un composant à un autre de la simulation. L'envoi d'un message stocke ce message dans une file maintenue par l'objet Environment. Les messages ne sont pas lus directement.

La mise à jour de tous les composants s'effectue ainsi :

- tous les objets Entity de la simulation sur lesquels la méthode `update()` est callable sont récupérés ;
- pour chacun de ces objets, la méthode `textttupdate()` est appelée ; dans le corps de cette méthode, des messages peuvent être envoyés d'un objet à un autre de la simulation ; ces messages sont stockés avec l'adresse des objets qui en sont les destinataires dans la file de l'objet Environment ;
- pour chaque élément (`<Destinataire>`, `<Message>`) de la file de l'objet Environment, appeler la méthode `handle_message()` sur l'objet `<Destinataire>` avec comme argument `<Message>`; d'autres messages peuvent être envoyés d'un objet à un autre de la simulation lors de l'exécution de cette méthode.

B. Communication entre le simulateur et l'application mobile client

La communication entre le simulateur et les applications Android passe par des requêtes HTML (ce que d'autres rapports ont appelés "l'API REST"). La communication entre le simulateur et les applications Android passe par des requêtes HTML GET et POST selon les cas, en utilisant ce que d'autres rapports ont appelé "l'API REST". Du côté de l'application Android client, les requêtes sont adressées à une adresse IP codée en dur dans le code Kotlin, qui peut toutefois être changée dans l'application via un menu caché (double-cliquer sur l'icône Urbanloop dans les paramètres, puis entrer un mot de passe). Quand l'application s'exécute dans un simulateur intégré à Android Studio, l'environnement de développement que le développeur original ainsi que notre groupe ont utilisé, le système d'exploitation hôte est accessible via d'adresse IP 10.0.2.2. Le port utilisé est 8090 (un port arbitraire parmi ceux ne nécessitant pas de droits d'administrateur, et n'étant pas canoniquement associé à un service existant), également codé en dur dans l'application et le simulateur. Du côté du simulateur Python, le module Flask est utilisé pour réceptionner, interpréter et répondre aux requêtes HTML reçues sur le port 8090. Les requêtes reçues et les réponses envoyées sont manipulées sous la forme d'objet ayant la structure de fichiers JSON. Le module Flask communique avec la simulation au moyen de la classe Network, dont il existe une instance par simulation en cours.

Liste des requêtes (avec arguments, valeurs de retour et actions) utilisées par l'application Android client

- Requête GET sur `/capsule/` suivi d'un identifiant d'utilisateur Cette requête est utilisée par l'application Android client à intervalles réguliers quand l'onglet "Trajet" est ouvert : la réponse à cette requête contient des informations sur l'état de la capsule dans laquelle circule actuellement l'utilisateur, notamment le temps avant d'arriver à destination.
- Requête POST sur `/new_trip` Cette requête est utilisée par la borne en station, qui réagit à la présentation d'un QR code issu de l'application Android client. Cette requête lance un nouveau trajet en spécifiant la station de départ (information connue de la borne), la station d'arrivée

(information issue du QR code), ainsi que le numéro d'utilisateur (issu du QR code, connu via le compte qui s'est connecté à l'application).

- Requête POST sur `/change_dest` Cette requête est lancée par l'application client quand on demande un changement de destination. Sont communiqués l'identifiant utilisateur et la nouvelle destination
- Requête GET sur `/emergency_exit` avec un identifiant d'utilisateur Cette requête déclenche un procédure d'urgence sur la capsule dans lequel circule l'utilisateur
- Requête POST sur `/capsule_revision` avec un numéro de capsule Cette requête a été ajoutée pendant ce PIDR. Cette requête entraîne l'ajout d'un signalement supplémentaire sur la capsule concernant son état mécanique. De plus, ce signalement, issu de l'API et non un signalement aléatoire généré dans le simulateur lui-même, est considéré comme prioritaire, et entraînera l'envoi de la capsule en révision même si le quota de signalement n'est pas encore atteint.
- Requête POST sur `/capsule_lavage` avec un numéro de capsule Cette requête a été ajoutée pendant ce PIDR. Cette requête entraîne l'ajout d'un signalement supplémentaire sur la capsule concernant son état de propreté. De plus, ce signalement, issu de l'API et non un signalement aléatoire généré dans le simulateur lui-même, est considéré comme prioritaire, et entraînera l'envoi de la capsule au lavage même si le quota de signalement n'est pas encore atteint.
- Requête POST sur `/envoi_revision` avec un numéro d'utilisateur Cette requête a été ajoutée pendant ce PIDR, et est identique à `/capsule_revision` à ceci près que l'argument n'est pas le numéro de capsule, mais le numéro de voyageur (à qui a été alloué la capsule à signaler), seul numéro connu par l'application client.
- Requête POST sur `/envoi_lavage` avec un numéro d'utilisateur Cette requête a été ajoutée pendant ce PIDR, et est identique à `/capsule_lavage` à ceci près que l'argument n'est pas le numéro de capsule, mais le numéro de voyageur (à qui a été alloué la capsule à signaler), seul numéro connu par l'application client.

Les modifications que nous avons faites dans l'application Android sont l'ajout de deux boutons de signalement, l'un pour la propreté, et l'autre pour l'état de fonctionnement mécanique de la capsule, dans l'onglet "Trajet", celui qui permet à l'utilisateur de suivre l'avancée de la capsule qui lui a été allouée.

C. Structure d'un réseau

Nous présentons ici la structure élémentaire des composants d'un réseau utilisé par le simulateur, dans son état avant le PIDR [4]. Nous utilisons les noms des classes utilisées en Python. Le réseau de la figure 7 correspond à la ressource `test_1.json` du dépôt. Un réseau est constitué d'une ou plusieurs boucles. Celui de la figure 7 en présente deux. Une boucle correspond à une succession d'objets Roads, chaque objet Road est relié au suivant par un objet Switch. Les objets Switch sont les disques violets de la figure 7. Les objets

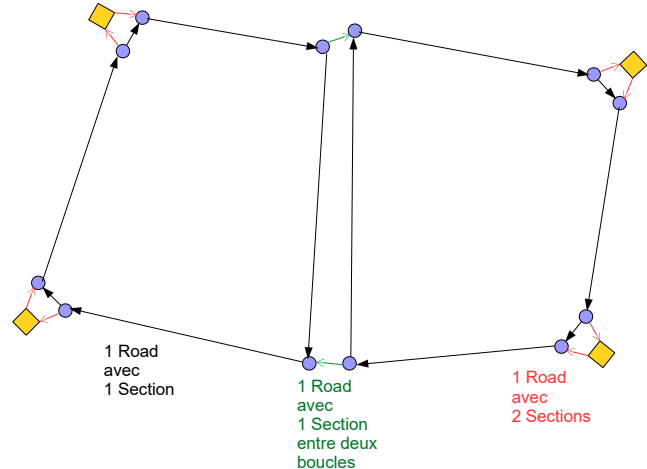


Fig. 1. Schéma simplifié d'un réseau

Roads sont les flèches noires. Les objets Switch peuvent être de deux types : les objets SwitchIn et les objets SwitchOut :

- un objet Switch dont le nombre de flèches entrantes est deux et le nombre de flèches sortantes est un est un objet SwitchIn ;
- un objet Switch dont le nombre de flèches sortantes est deux et le nombre de flèches entrantes est un est un objet SwitchOut.

Le cheminement des pods sur une boucle s'effectue dans un seul sens, représenté par le sens des flèches sur la figure 7. Ainsi, deux boucles A et B sont reliées en deux points, par un objet Road d'un SwitchOut de A vers un SwitchIn de B, et par un objet Road d'un SwitchOut de B vers un SwitchIn de A. Ce type de liaison est représenté par les flèches vertes de la figure 7.

Dans un réseau sont aussi présents les objets Step, représentés par les losanges jaunes en figure 7. Ce sont les objets Shed (les hangars des capsules), les Stations (les stations permettant aux voyageurs de monter dans les capsules) et les objets Sensor (permettant d'effectuer des tests dans le simulateur).

Un objet Step est relié au réseau grâce à un objet Road particulier. Tous les objets Road mentionnés précédemment contenaient chacun un seul objet de type Section (qui matérialise concrètement une route du réseau). En revanche, l'objet Road qui lie un objet Step à une boucle contient deux objets Section, représentés en figure 7. L'un de ces objets Section a pour origine un SwitchOut de la boucle, et pour destination l'objet Step. L'autre a pour origine l'objet Step, et pour destination un SwitchIn de la boucle.

D. Fonctionnalités partiellement implantées du simulateur

Avant de commencer notre projet, nous avons identifié avec l'aide de M. Cholez les différentes fonctionnalités du simulateur non opérationnelles.

1) Sauvegarde et rechargement d'un réseau:

Le réseau d'une simulation en cours de lancement peut être sauvegardé dans un fichier JSON, mais le rechargement de ce fichier occasionne une erreur.

2) Calcul du plus court chemin:

Les capsules prennent le plus court chemin entre leur départ et leur destination, et quand elles choisissent elle-même leur destination (par exemple, quand il faut choisir la station de lavage), elles choisissent la destination possible qui est la plus proche de leur station de départ.

3) Fonctionnement des comptes des utilisateurs:

Les comptes d'utilisateurs n'utilisent pas encore une vraie base de données centralisée. Actuellement il est possible de se connecter avec n'importe quel couple adresse électronique / mot de passe, et le numéro de voyageur associé sera toujours le numéro 123.

III. OBJECTIFS ET EXIGENCES

Dans le réseau, les pods peuvent seulement partir d'un Shed vers un objet Station pour approvisionner la station, d'un objet Station vers un autre objet Station pour transporter un voyageur ou pour réguler le nombre de capsules en station, et d'un objet Station vers un objet Shed également pour réguler le nombre de capsules en station.

Cependant, dans un réseau urbain réaliste, les pods doivent pouvoir être révisés ou réparés sans avoir à être retirés complètement du réseau. Ils doivent aussi pouvoir être nettoyés rapidement. La révision et le lavage ne doivent en outre pas perturber le trafic des autres capsules.

Ce projet a comme objectifs principaux :

- la conception d'un dispositif d'envoi des capsules dans de nouveaux types d'entrepôts (de révision et de lavage), de traitement des capsules dans ces nouveaux entrepôts et de réintégration des capsules dans les stations du réseau
- le choix d'une méthode permettant de demander au simulateur l'exécution de ce nouveau dispositif sur une capsule
- l'initialisation aléatoire des attributs des pods concernant leur état
- le signalement aléatoire d'un pod de la part de son voyageur.

IV. SOLUTIONS

Il a été convenu que les deux types d'évènements suivants pouvaient être effectués :

- la décision du simulateur d'envoyer ou non une capsule en révision ou au lavage
- le signalement de la part d'un voyageur (donc d'un objet Traveler du simulateur ou d'un utilisateur de l'API Rest) d'une capsule comme étant à laver ou à réviser.

A. Premières solutions

Nous avons d'abord imaginé les solutions suivantes. Toutes n'ont pas été conservées. La liste des solutions conservées et implantées se trouve en IV-B.

1) Lavage:

Un envoi au lavage peut être demandé sur signalement d'un passager, mais aussi de façon automatique, et selon des critères comme la météo ou un évènement particulier. L'entrepôt de lavage sélectionné doit être le plus proche ou le moins encombré. Plusieurs entrepôts de lavage peuvent être présents dans le réseau.

2) Révision:

L'envoi en révision est automatiquement déclenché lorsqu'une capsule a parcouru une certaine distance limite ou qu'elle a voyagé pendant une certaine durée limite depuis la dernière révision. Un échelonnage des entretiens devrait permettre d'éviter un trop grand nombre de capsules en révision. Une capsule peut être affectée à un entrepôt d'entretien spécifique.

Des réparations inhabituelles doivent être prévues, comme par exemple lorsqu'un accident a lieu, ou lorsqu'une capsule détecte un choc.

Les voyageurs doivent également pouvoir signaler une capsule comme étant à réviser.

3) Limites aux signalements:

- au premier signalement, envoyer la capsule (qui doit être vide) à une autre station proche, et ne l'envoyer au lavage ou en révision qu'après le second signalement
- valider une demande d'envoi si deux signalements ont été faits par deux voyageurs différents sur la même capsule
- garder en mémoire les utilisateurs connus comme étant mal intentionnés
- limiter le nombre de signalements par compte, par jour et par station.

B. Solutions retenues

Au vu de la structure du simulateur, nous avons d'abord retenu les objectifs suivants :

1) Remplacer la classe Shed par les nouveaux types de hangars:

- remplacer la classe Shed par une classe abstraite Hangar, et étendre cette classe avec HangarSimple (pour un Shed classique), HangarRevision pour un entrepôt de révision, et HangarLavage pour un entrepôt de lavage ;
- concevoir deux classes GestionnaireRevision et GestionnaireLavage contenant les informations nécessaires à un réseau pour organiser l'entretien et le lavage.

2) Modifier l'état d'une capsule selon différentes méthodes:

- un utilisateur du simulateur peut signaler une capsule comme étant à laver ou à réviser
- un objet Traveler sortant d'une capsule signale aléatoirement une capsule comme étant à laver ou à réviser
- la distance parcourue d'une capsule depuis sa dernière révision est mise à jour au franchissement de chaque Switch
- la durée écoulée depuis la dernière révision d'une capsule est mise à jour à chaque fois que la méthode update () de l'objet Pod est appelée

3) *Placer les informations relatives à l'état de la capsule dans la capsule elle-même:*

La capsule contient :

- le nombre de signalements reçus pour l'envoyer au lavage
- le nombre de signalements reçus pour l'envoyer en entretien
- son temps restant de révision si elle est en cours de révision
- son temps restant de lavage si elle est en cours de lavage
- sa distance parcourue depuis sa dernière révision
- le temps écoulé depuis sa dernière révision
- un indicateur stipulant qu'elle a été signalée comme devant aller au lavage par un vrai utilisateur (par l'API Rest) et non pas par un objet Traveler
- le même type d'indicateur, mais pour aller en révision
- un booléen indiquant si la capsule est en direction d'un HangarLavage
- un booléen indiquant si la capsule est en direction d'un HangarRevision

4) *Placer les informations relatives aux caractéristiques de lavage et de révision dans des gestionnaires:* Deux gestionnaires d'un réseau, GestionnaireLavage et GestionnaireRevision, contiennent les nombres limites de signalements avant révision ou lavage, les distances maximales et durées maximales avant révision.

5) *Observer l'état de la capsule à chaque fois qu'elle arrive à destination:*

Lorsqu'un pod :

- arrive à destination et que cette destination est un HangarSimple ou une Station
- termine sa révision ou son lavage

il est demandé aux gestionnaires du réseau si la capsule doit ou non aller en lavage et/ou en révision. La priorité est donnée au lavage. Ce choix a été fait pour observer un nombre de capsules en direction d'un HangarLavage et un nombre de capsules en direction d'un HangarRevision similaires lors des tests effectués. Si la capsule est marquée comme étant à laver ou à réviser, elle est orientée selon ce marquage en sortie de station ou de hangar.

6) *Empêcher la réquisition de capsules en HangarSimple:*

Les stations manquant de capsules peuvent demander des pods supplémentaires à un HangarSimple. Il ne faut cependant pas que les pods envoyés soient à laver ou à réviser. Il faut donc empêcher le HangarSimple d'envoyer ce genre de pods usagés. C'est pourquoi, comme nous l'expliquons en IV-B5, l'état d'un pod est aussi observé en HangarSimple, qui envoie un pod usagé dans un hangar adéquat.

7) *Permettre un signalement aléatoire:*

À la sortie d'un pod, un objet Traveler signale aléatoirement le pod comme étant à réviser ou à laver. Cela permet de visualiser l'effet des apports de ce PIDR sur l'état à long terme du simulateur sans avoir à interagir trop souvent avec l'API Rest.

8) *Permettre une initialisation aléatoire:*

Les attributs des pods créés concernant leur état technique et hygiénique sont initialisés aléatoirement. Cela permet d'éviter

un départ simultané de capsules en révision ou en lavage, ce qui perturberait le réseau.

9) *Adapter à tous ces changements les composants JavaScript:*

La partie graphique du simulateur doit prendre en compte les changements apportés au mécanisme de simulation.

V. NOUVELLE ARCHITECTURE

Avant-propos

Nous souhaitons avant tout préciser que dans la plupart des classes que nous avons créées ou modifiées, est présent l'attribut

```
self._quickInfos
```

et la méthode

```
self.setQuickInfos()
```

Cet attribut permet d'enregistrer directement dans une chaîne de caractères les informations d'une classe à afficher par l'objet JavaScript Hint. Il est mis à jour à l'initialisation d'un objet de la classe et au début de chaque appel de `update()` sur cet objet. Cela permet d'obtenir efficacement les informations de la classe directement accessibles en Python, sans avoir à ajouter d'attributs ou d'appels de méthodes en JavaScript. Nous ne mentionnons pas cet attribut par la suite.

Enfin, pour chaque attribut ajouté dans les classes à convertir en dictionnaire au format JSON, nous avons ajouté les instructions nécessaires dans les méthodes `serialize()` de chacune de ces classes.

A. Nouveaux types d'entrepôts

La classe Shed n'est plus utilisée. La nouvelle classe abstraite Hangar est étendue par la classe HangarSimple (le nouveau Shed), la classe HangarRevision et la classe HangarLavage.

1) Classe Hangar:

Cette classe contient (en plus des informations de sa classe mère Step) les informations communes à tous les types d'entrepôts.

```
self._capacity
self._element_of_loop
self._pods
self._departure_pods
```

2) Classe HangarSimple:

Cette classe contient (en plus des informations de sa classe mère Hangar) :

```
self._doiventAllerEnRevision
self._doiventAllerAuLavage
```

Ce sont deux attributs permettant au HangarSimple de conserver les références des pods devant aller en lavage ou en révision.

La méthode `update()` a été modifiée par rapport à l'ancien entrepôt Shed. Elle procède ainsi:

- `quickInfos` est mis à jour

- le temps avant le prochain départ est mis à jour
- pour chaque pod dans `self._pods`, il est demandé à l'objet `GestionnaireLavage` du réseau et à l'objet `GestionnaireRevision` si le pod doit aller au lavage ou en révision, et :

- le pod est mis dans la liste
`self._doiventAllerAuLavage` s'il doit aller au lavage et qu'il n'a pas encore été mis dans `self._doiventAllerAuLavage` ou dans `self._doiventAllerEnRevision`
- le pod est mis dans la liste
`self._doiventAllerEnRevision` s'il doit aller au lavage et qu'il n'a pas encore été mis dans `self._doiventAllerAuLavage` ou dans `self._doiventAllerEnRevision`

on note que la priorité est donnée au lavage

- l'organisation du départ des capsules s'effectue comme dans l'ancien Shed

La méthode `handle_message()` a été modifiée par rapport à l'ancien entrepôt Shed. Elle procède ainsi:

- un message "pod_entry" est traité comme pour Shed
- un message "pod_exit" est traité comme pour Shed, hormis le fait que le pod sortant est retiré de `self._doiventAllerAuLavage` ou de `self._doiventAllerEnRevision` s'il y est présent
- un message "refill" est traité comme pour Shed, hormis le fait que si un pod disponible est trouvé mais qu'il doit aller en révision ou au lavage, le `HangarSimple` répond à la station demandeuse de pods qu'elle n'en recevra pas de lui.

3) Classe *HangarRevision*:

Cette classe contient (en plus des informations de sa classe mère `Hangar`) :

`self._enRevision`

C'est une liste des pods en cours de révision.

La méthode `update()` a été modifiée par rapport à l'ancien entrepôt Shed. Elle procède ainsi:

- `quickInfos` est mis à jour
- le temps avant le prochain départ est mis à jour
- pour chaque pod dans `self._enRevision`, le temps restant en révision est mis à jour, et si le pod a terminé sa révision, ses attributs

```
nombre_signalements_doit_aller_
    \_en_revision
dernier_signalement_revision_par_vrai_
    \_utilisateur
temps_depuis_revision
distance_depuis_revision
temps_restant_attente_en_revision
```

sont mis à jour

- le pod en tête des pods qui doivent partir est orienté vers un `HangarLavage` obtenu par l'objet `GestionnaireLavage`

du réseau si le pod est à laver, sinon le pod est orienté vers un `HangarSimple` obtenu par l'objet `GestionnaireRevision` du réseau.

La méthode `handle_message()` a été modifiée par rapport à l'ancien entrepôt Shed. Elle procède ainsi:

- un message "pod_entry" ou "pod_exit" est traité comme pour Shed
- un message "refill" n'est pas traité car il n'a pas été prévu qu'une station puisse demander des pods en renfort à un entrepôt de révision.

4) Classe *HangarLavage*:

Le contenu de cette classe est le même que pour la classe `HangarRevision`, hormis les faits suivants :

- la classe contient l'attribut `self._enLavage` au lieu de `self._enRevision`
- dans `update()`, ce sont les attributs

```
pod.nombre_signalements_doit_aller_
    \_au_lavage
pod.dernier_signalement_lavage_par_
    \_vrai_utilisateur
pod.temps_restant_attente_au_lavage
```

qui sont mis à jour

- le pod en tête des pods qui doivent partir est orienté vers un `HangarRevision` obtenu par l'objet `GestionnaireRevision` du réseau si le pod est à réviser, sinon le pod est orienté vers un `HangarSimple` obtenu par l'objet `GestionnaireLavage` du réseau.

B. Création des gestionnaires

1) *Gestionnaire*: Cette classe abstraite contient la méthode `obtenirDestination()`. À partir d'une station source et d'une catégorie de destination (`HangarSimple`, `HangarRevision`, `HangarLavage`), cette méthode obtient une destination du type spécifié et la plus proche de la station source. L'algorithme de Dijkstra, implémenté dans les projets précédents, est utilisé.

2) *GestionnaireRevision*: Cette classe étend la classe abstraite `Gestionnaire`. Elle contient les attributs suivants:

```
_limite_distance_parcourue_avant_revision
_limite_temps_ecoule_avant_revision
_limite_nombre_signalements_avant_revision
_temps_revision = temps_revision
```

Les trois premiers sont utilisés par la méthode `podDoitAllerEnRevision()` pour déterminer si un pod doit aller en révision : si l'un de ces trois attributs est plus petit que l'attribut correspondant du pod, ou si le pod a été signalé par un vrai utilisateur (et non pas aléatoirement par un objet `Traveler`), alors cette méthode informe son appelant que le pod doit aller en révision. Le dernier attribut, `_temps_revision`, permet à un `HangarRevision` de connaître le temps que met un pod du réseau à être révisé.

Les méthodes de classe

```

genererNombreSignalementsAleatoire()
genererDistanceAleatoire()
genererTempsAleatoire()

```

sont utilisées lors de l'initialisation d'un pod dont les informations ne sont pas dans le fichier JSON du réseau. Cela permet d'obtenir un pod déjà usagé.

3) *GestionnaireLavage*: Cette classe étend la classe abstraite *Gestionnaire*. Elle contient les attributs suivants:

```

_limite_nombre_signalements_avant_lavage
_temps_lavage = temps_lavage

```

Le premier attribut est utilisé par la méthode `podDoitAllerAuLavage()` pour déterminer si un pod doit aller au lavage : si cet attribut est plus petit que l'attribut correspondant du pod, ou si le pod a été signalé par un vrai utilisateur (et non pas aléatoirement par un objet *Traveler*), alors cette méthode informe son appelant que le pod doit aller au lavage. Le dernier attribut, `_temps_lavage`, permet à un *HangarLavage* de connaître le temps que met un pod du réseau à être nettoyé.

La méthode

```

genererNombreSignalementsAleatoire()

```

est utilisée lors de l'initialisation d'un pod dont les informations ne sont pas dans le fichier JSON du réseau. Cela permet d'obtenir un pod déjà usagé.

C. Modifications des classes existantes

1) *Classe Pod*:

Les attributs suivants sont ajoutés à la classe *Pod*:

```

self._nombre_signalements_doit_aller
    \_en_revision
self._dernier_signalement_lavage_par_
    \_vrai_utilisateur
self._temps_depuis_revision
self._distance_depuis_revision
self._temps_restant_attente_en_revision
self._en_direction_revision
self._nombre_signalements_doit_aller_
    \_au_lavage
self._dernier_signalement_revision_par_
    \_vrai_utilisateur
self._temps_restant_attente_au_lavage
self._en_direction_lavage

```

Notons que si les attributs

```

_nombre_signalements_doit_aller_en_revision
_temps_depuis_revision
_distance_depuis_revision
_nombre_signalements_doit_aller_au_lavage

```

ne sont pas présents dans le fichier JSON chargé, ils sont initialisés aléatoirement à l'aide des gestionnaires du réseau. Cela permet de générer des capsules avec des nombres de signalements aléatoires et déjà plus ou moins usagés.

Dans la méthode `update()` de *Pod*, il a seulement été ajouté l'incrémentation de l'attribut `_temps_depuis_revision`.

2) *Classe Traveler*:

Les méthodes `signalerAleatoirementLavage()` et `signalerAleatoirementRevision()` sont utilisées par la classe *Station*.

3) *Classe Station*:

Nous souhaitons d'emblée remarquer que l'initialisation, le traitement et l'enregistrement de l'attribut `self._departure_pods` ont été modifiés pour parachever le travail des groupes précédents. En effet, le rechargement des fichiers JSON sauvegardés lors d'une simulation en cours déclenchait une erreur avant le début du projet, et l'un des problèmes causant cette erreur était la mauvaise gestion de cet attribut. Nous avons tenté de corriger le problème, et désormais un fichier JSON peut être rechargé, mais le réseau résultant n'est pas totalement fonctionnel :

- Les pods devraient être recréés dans les listes maintenues par le fichier `statistiques.py`, qui ne conserve que des références de pods
- Des informations sur les voyageurs, comme leur destination, ne sont pas encore conservées dans les fichiers JSON, donc le rechargement crée des voyageurs sans destination

Les ajouts faits dans la classe *Station* sont très proches de ceux faits dans *HangarSimple* par rapport à l'ancienne classe *Shed*. Cette classe contient les attributs supplémentaires :

```

self._doiventAllerEnRevision
self._doiventAllerAuLavage

```

Ce sont deux attributs permettant de conserver les références des pods devant aller en lavage ou en révision.

La méthode `update()` a été modifiée. Elle procède ainsi:

- pour chaque pod dans `self._pods`, il est demandé à l'objet *GestionnaireLavage* du réseau et à l'objet *GestionnaireRevision* si le pod doit aller au lavage ou en révision, et :
 - le pod est mis dans la liste `self._doiventAllerAuLavage` s'il doit aller au lavage et qu'il n'a pas encore été mis dans `self._doiventAllerAuLavage` ou dans `self._doiventAllerEnRevision`
 - le pod est mis dans la liste `self._doiventAllerEnRevision` s'il doit aller au lavage et qu'il n'a pas encore été mis dans `self._doiventAllerAuLavage` ou dans `self._doiventAllerEnRevision` on note que comme pour *HangarSimple*, la priorité est donnée au lavage
- les passagers ne montent que dans les pods qui ne sont ni à laver ni à réviser
- pour chaque pod à laver ou à réviser, une destination est obtenue grâce au gestionnaire correspondant du réseau, et le pod est orienté vers cette destination

- le temps de départ entre deux capsules est mis à jour
- lorsqu'une capsule quitte la station, elle est retirée de la liste `self._doiventAllerEnLavage` ou `self._doiventAllerEnRevision` si elle y est présente

La méthode `handle_message()` a été modifiée. Elle procède ainsi:

- le traitement d'un message "pod_entry" ne change pas, hormis le fait que le passager sortant de la capsule signale ou non la capsule comme étant à laver ou à réviser (ceci est fait aléatoirement)
- le traitement d'un message "pod_exit" ne change pas
- le traitement d'un message "refill" ne change pas, hormis le fait que si un pod vide qui n'est pas sur le point de partir est trouvé mais qu'il doit aller en révision ou au lavage, la station ne le libère pas.

4) Classes *SwitchIn* et *SwitchOut*:

Une seule modification a été apportée à chacune de ces classes. Lorsqu'un pod quitte un *SwitchIn* ou un *SwitchOut* (donc lorsque le switch reçoit un message "pod_exit"), la longueur du dernier objet *Section* contenu dans l'objet *Road* qui précède le switch, est ajoutée à la valeur de l'attribut `_distance_depuis_revision` du pod. Cela permet de mettre à jour la distance parcourue par le pod à chaque fois qu'il franchit un switch.

VI. CONCLUSION

L'application peut désormais communiquer avec le simulateur. Un utilisateur peut donc suivre en temps réel les informations concernant son trajet (station précédente, prochaine station).

Après avoir obtenu son QR Code et l'avoir scanné avec la borne, l'utilisateur peut signaler ce pod comme à laver ou à réviser. Le simulateur prend alors en compte ce signalement, ce qui est visible par l'info-bulle affichée lorsqu'un pod est pointé, et par les changements de couleur des pods après l'arrivée de ceux-ci.

Les actions de l'utilisateur peuvent aussi être faites avec l'API Rest.

Un gestionnaire de révision considère qu'un pod est à réviser s'il a parcouru une distance supérieure à la distance maximale, ou s'il a circulé pendant une durée supérieure à la limite du réseau, ou s'il a été signalé par un voyageur virtuel un nombre de fois supérieur à une limite, ou s'il a été signalé par un voyageur réel. Un gestionnaire de lavage considère qu'un pod est à laver s'il a été signalé par un voyageur virtuel un nombre de fois supérieur à une limite, ou s'il a été signalé par un voyageur réel.

Enfin, les voyageurs virtuels du simulateur signalent aléatoirement la capsule dont ils sortent comme à laver ou à réviser, permettant d'évaluer les effets de cette nouvelle possibilité de signalement sur le fonctionnement et les performances du système Urbanloop.

REMERCIEMENTS

Nous remercions M. Cholez pour son suivi très régulier et pour l'aide et les explications qu'il nous a apportées lorsque nous rencontrons des difficultés. Nous souhaitons également remercier M. Lang pour son excellente présentation du projet Urbanloop et du simulateur, qui nous a permis d'avoir une vue d'ensemble du projet dès le commencement de note PIDR.

REFERENCES

- [1] Lilian Bidet, *Rapport de stage - Extension de l'application mobile Urbanloop*, 2019-2020.
- [2] Pierre-Antoine Grosselin, *Rapport de stage - Fiabilisation et exploitation du simulateur d'UrbanLoop*, 2019-2020.
- [3] Clément Crouzet, Tristan Deniau et Arthur Saout, *Rapport de projet industriel - Réalisation d'une borne de station Urbanloop et exploitation du simulateur*, 2020-2021.
- [4] Tristan Le Godais et Malo Mongard, *Proposition de terminologie du réseau de transport UrbanLoop*.

APPENDIX

Le contenu des annexes se trouve ci-dessous.

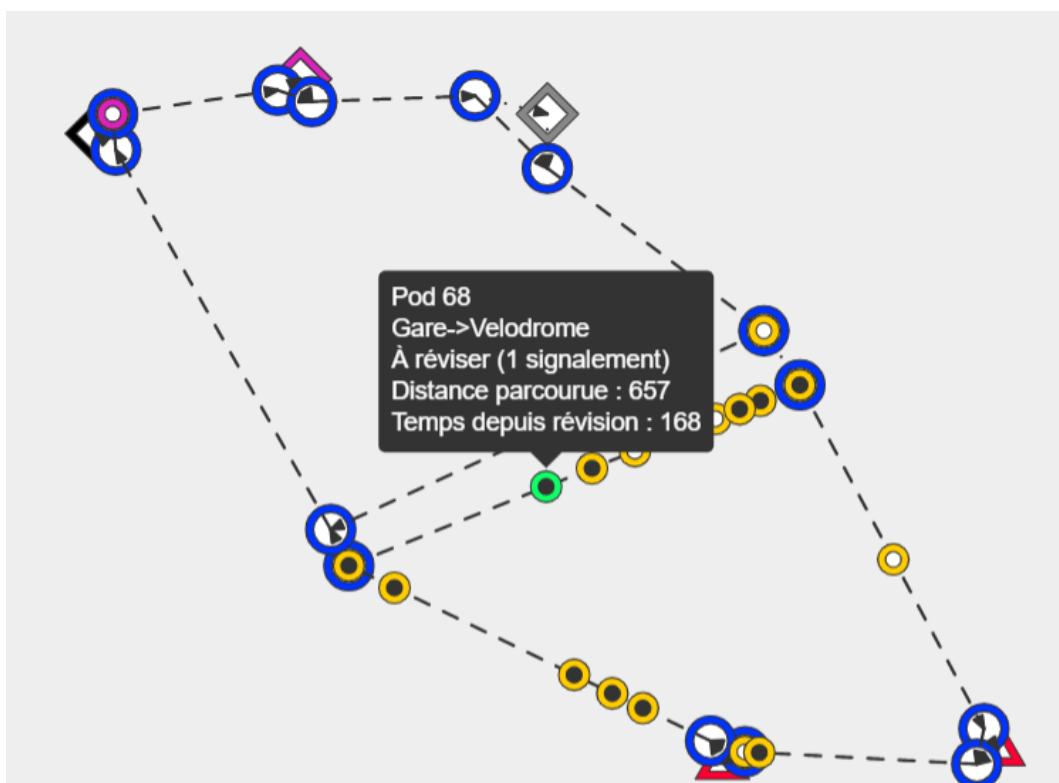


Fig. 2. Pod en demande de révision

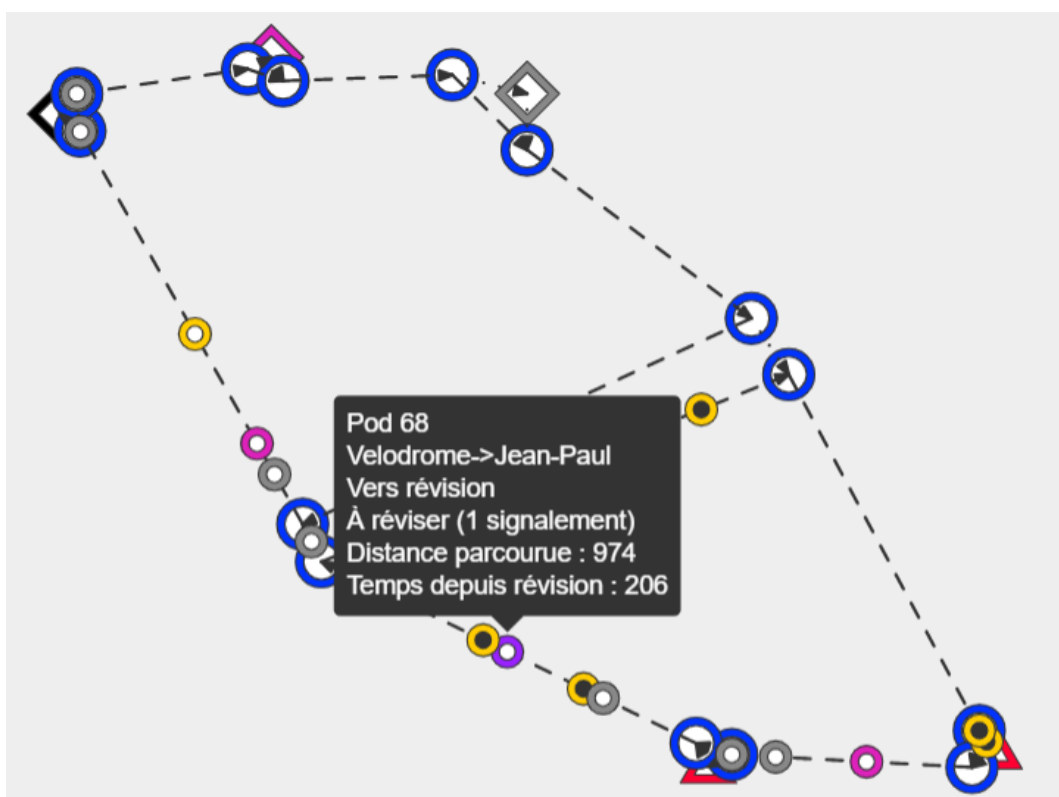


Fig. 3. Pod en direction d'un entrepôt de révision

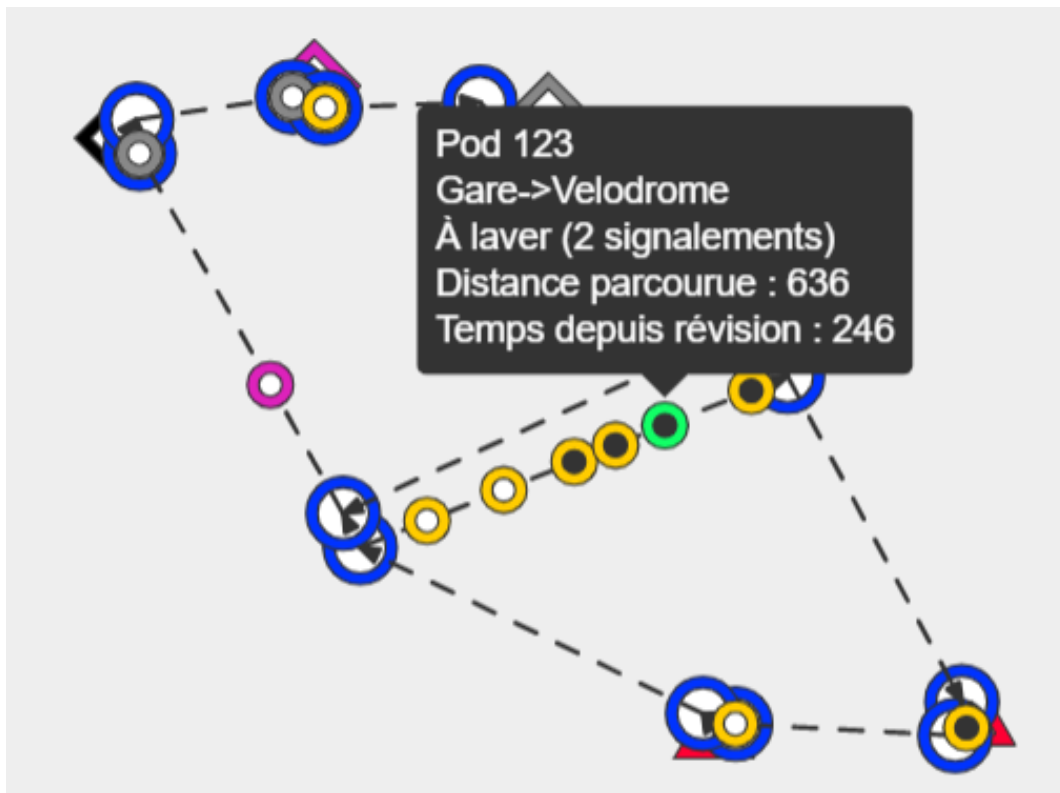


Fig. 4. Pod en demande de lavage

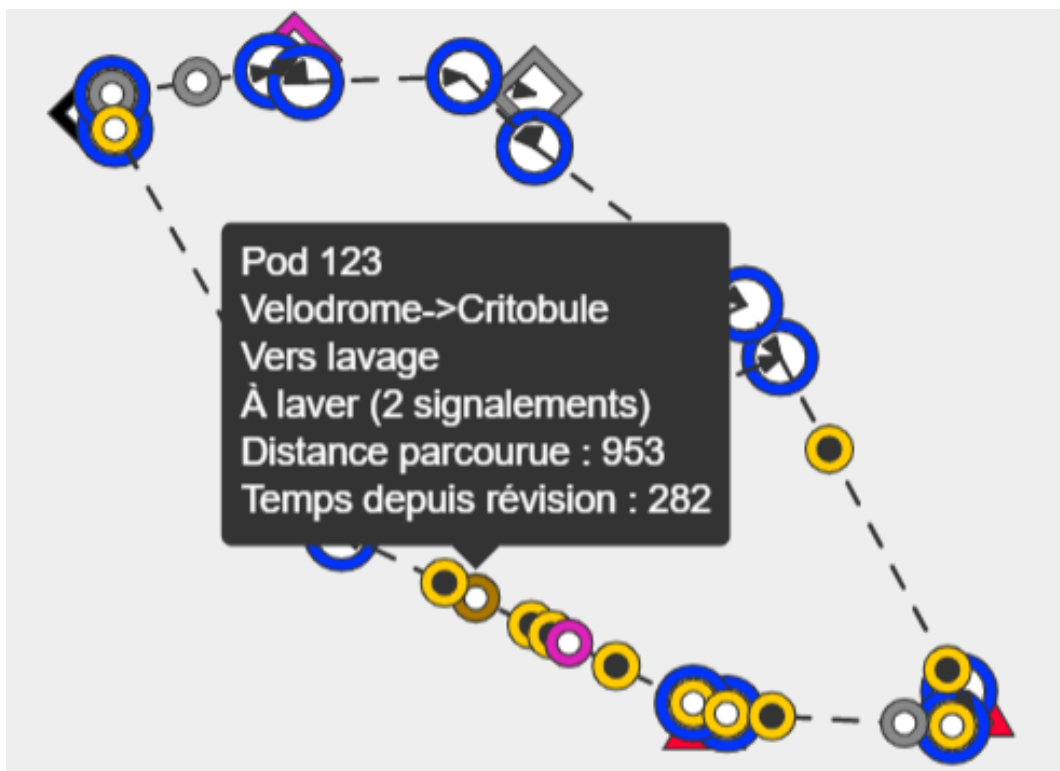


Fig. 5. Pod en direction d'un entrepôt de lavage

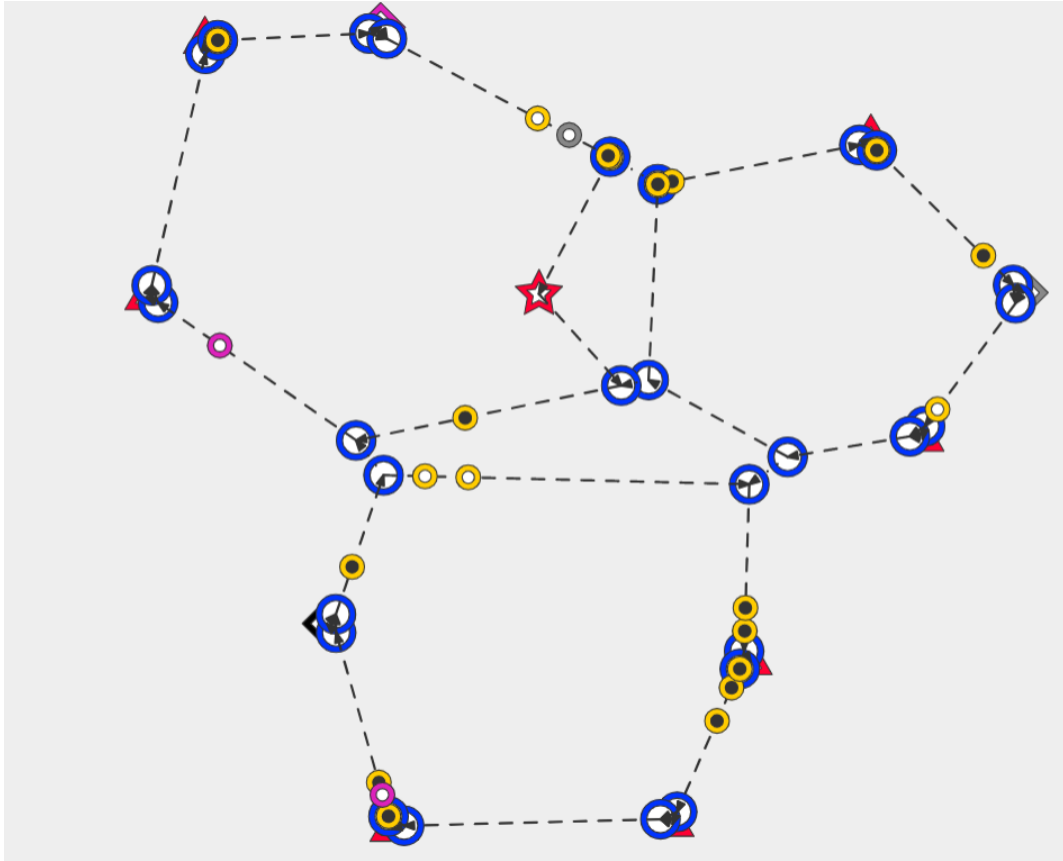


Fig. 6. Réseau mini_network en cours de simulation



Fig. 7. Vue de l'onglet trajet de l'application